

Evading the Cloud Tax: Beating the 'Storage Full' Notification

Deep Learning Mini-Project — Technical Report

1. System Overview

We repurpose an **unmodified ResNet-101** as a binary storage medium by reinterpreting its ~44.55 million float32 parameters (170.4 MB on disk) as a flat byte array. Images are Standard JPEG-compressed (quality 95, 4:4:4 chroma) and their raw byte-streams are **bit-cast directly into parameter tensors**. The final fully-connected layer (fc.weight, 1000×2048) is reserved as a compact metadata header; all remaining float32 tensors provide **162.5 MB of usable payload**.

2. Bit-Packing Strategy

2.1 Data Representation — Standard JPEG (4:4:4)

Each source image is converted to RGB and encoded with **Standard JPEG at quality=95 and subsampling=0** (full 4:4:4 chroma retention). This eliminates chroma sub-sampling artifacts and achieves an average PSNR of **56.8 dB** versus the camera originals, while compressing 12 MP (4032×3024) images from 34.9 MB raw to 1–3 MB — a 12–35× reduction. JPEG-2000 and PNG were excluded per project constraints.

2.2 Float32 Byte-Cast (zero quantisation)

Each IEEE 754 float32 occupies exactly 4 bytes. We reinterpret the JPEG byte-stream directly as float32 values using numpy's `.view(np.float32)` — no normalisation, no truncation, no FP16 down-cast. This gives a perfect **4:1 byte-to-parameter ratio** and guarantees that the recovered JPEG stream is **bit-for-bit identical** to the encoded stream (measured PSNR between encode output and decode output is ∞ dB / MSE = 0). Any PSNR degradation versus the ground truth is attributable solely to the JPEG codec, not to the vault mechanism.

2.3 Tensor Mapping Strategy

All float32 entries in the `state_dict` — spanning `conv1`, `bn1`, and `layer1–layer4` (conv weights, BN weight/bias/running_mean/running_var) — are collected and **sorted alphabetically by key**. This deterministic ordering is used identically in `encode.py` and `decode.py`, eliminating any need for an external index. The concatenated JPEG payload fills these tensors sequentially in C-contiguous (row-major) order; remaining capacity is zero-padded. For 4D conv tensors ($C_{out}, C_{in}, K_H, K_W$), bytes map naturally along the innermost dimension first.

2.4 Metadata Header — fc.weight

The `fc.weight` tensor (1000×2048 float32 = 8.19 MB) stores a binary header with the following little-endian layout:

Offset	Size	Field
0	4 B	Magic sentinel: ASCII 'VALT'
4	4 B	num_images (uint32)
8 + 4i	4 B	jpeg_length[i] (uint32), i = 0 ... N-1
...	—	Zero-padding to fill remaining fc.weight bytes

Header capacity: $(8\,192\,000 - 8) / 4 = 2\,047\,998$ image slots — far beyond any physical storage limit.

3. Quantisation Approach

We use a **zero-quantisation** strategy: JPEG bytes are stored verbatim inside 32-bit floats with no bit-truncation, no FP16 down-cast, and no INT8 rounding. Reducing to FP16 would corrupt the lower 16 bits of every parameter that encodes a non-trivial byte pair, halving usable capacity while introducing systematic data corruption. Our approach

treats the model file as a raw binary blob, using the .safetensors format's bit-exact serialisation to guarantee lossless round-trips through disk.

4. Evaluation Results

Metric	Value	Notes
Storage Efficiency E	0.9529	JPEG payload (162.4 MB) ÷ vault file (170.4 MB)
Capacity C	87 images	JPEG q=95; 162.5 MB payload capacity; 200 source images tested
Avg PSNR vs ground truth	56.80 dB	Decoded image vs camera original (86-image sample)
Min PSNR vs ground truth	51.54 dB	Worst-case image in corpus
Stream round-trip fidelity	Lossless (∞ dB)	JPEG bytes recovered bit-for-bit
JPEG quality / subsampling	95 / 4:4:4 (subsampling=0)	No chroma loss
Header overhead	8.19 MB	fc.weight; fc.bias left as zeros
Vault file size	170.4 MB	Full float32 ResNet-101 state_dict
Quantisation	None (FP32)	Bytes stored verbatim in IEEE 754 floats

Table 1. Performance on 200-image, 12 MP (4032×3024) test corpus. PSNR measured after full encode→save→load→decode pipeline.

5. Pipeline Design

encode.py

(1) Instantiate blank ResNet-101 via torchvision.models.resnet101(weights=None). (2) Collect and sort all float32 state_dict keys excluding fc. (3) JPEG-compress source images at q=95 / 4:4:4; skip images that would overflow the 162.5 MB capacity. (4) Reinterpret concatenated JPEG bytes as float32 via numpy.view(np.float32) and write into sorted tensors. (5) Encode the metadata header into fc.weight; save via safetensors.torch.save_file.

decode.py

(1) Load state_dict with safetensors.torch.load_file. (2) Read magic + JPEG lengths from fc.weight bytes. (3) Concatenate storage tensor bytes in the same alphabetical key order. (4) Slice N JPEG byte-streams by their stored lengths; decode each with PIL and write to output directory.

6. Trade-Off Analysis

The primary trade-off is between **capacity** and **PSNR**. Lowering JPEG quality to 85 roughly doubles capacity to ~156 images (avg ~1.04 MB/image) at the cost of ~8 dB PSNR (avg ~48.4 dB). Storing raw pixels (no compression, FP32) would be perfectly lossless (∞ PSNR) but limit capacity to **4 images** (162.5 MB ÷ 34.9 MB). Our chosen operating point (quality=95) maximises photographic PSNR among JPEG strategies while achieving 87× the capacity of raw storage. The zero-quantisation vault mechanism guarantees that the **storage layer itself introduces zero additional loss**: any PSNR gap versus the ground truth reflects the JPEG codec only.

All state_dict tensors retain valid float32 bit patterns. The vault is self-contained and decoded using only the loaded state_dict — no sidecar files or external indices are required.